# Fuzzy Logic Framework Overview (with FLBall3DAgent Example)

## Introduction

FLBall3DAgent demonstrates the fuzzy logic implementation to balance a ball on an agent's head. Note that this demo is the same as used in the Unity ML-Agents package, just modified to demonstrate Fuzzy Logic control.

The Fuzzy Logic Framework provides an API for implementing decision logic that operates on continuous variables using fuzzy inference. The `FLBall3DAgent` script illustrates how this framework integrates fuzzy reasoning to control behaviors such as head rotation to balance a ball. Reviewing this example is a great starting point to get up to speed before you begin the Racetrack assignment.

Open the `FL_3DBall` scene and refer to the `Assets/Scripts/FLBall3DAgent.cs` script.

The following highlights the fuzzy logic implementation in the `FLBall3DAgent`. However, you can also refer to *FuzzyLogicAPI_for_Racetrack.pdf* for further details of the Fuzzy Logic API.

### Defining Fuzzy Variables

Each fuzzy input and output variable is defined as an enumeration. The enum names identify linguistic categories such as *Negative*, *Zero*, and *Positive*. There is no requirement that there be exactly three categories, but that is a good rule of thumb (you need at least two). Start small and expand only when you identify a specific need.

```
public enum FzInputBallPosZ { Negative, Zero, Positive }
public enum FzOutputHeadRotX { RotNegDir, NoRot, RotPosDir }
```

### Constructing Fuzzy Sets

Each fuzzy variable requires a set of Degree-of-Membership (DoM) functions. These can be continuous (triangular, shoulder, or trapezoidal) or discrete. For inputs, prefer the crossfade helper `GenerateCrossfadeFuzzySet<T>`; for outputs, prefer `GenerateDiscreteFuzzySet<T>`.

**Example (input variable with crossfade functions).**

```
var posZSet = GenerateCrossfadeFuzzySet<FzInputBallPosZ>(
    (-1f, -0.2f), (0f), (0.2f, 1f) );
```

`GenerateCrossfadeFuzzySet` is a powerful helper method. For the Racetrack assignment, you should use this method rather than creating each DoM function manually. The arguments to the method coincide with the number of linguistic categories for the specified fuzzy variable (e.g., the number of enum values). Each argument provides one control point (float) or two control points (as a tuple of two floats), defining X-axis positions of the membership functions. The number you provide, as well as relative position, determine the actual DoM function modeled. The outermost functions (at either extreme) are always interpreted as shoulders. However, if only one control point is provided, then the

shoulder's plateau is shrunk down to a point. The interior arguments are either interpreted as triangles (one control point) or trapezoids (two control points). All DoM functions crossfade with their neighbors to the left and right. The closest control point of the neighbor in either direction indicates the crossfade control point.

Optionally, you may provide a second list specifying the peak or plateau DoM heights for each control point. The first list defines the X-axis positions (control points), and the second list defines the corresponding DoM heights. If omitted, all heights default to 1.0, which is recommended for nearly all use cases. Adjusting heights is mainly useful for emphasizing or de-emphasizing specific linguistic terms without changing their boundaries.

**Example (output variable with discrete functions).**

```
var fuzzySet = GenerateDiscreteFuzzySet<FzOutputHeadRotX>(
    -absMaxAngle, 0f, absMaxAngle );
```

`GenerateDiscreteFuzzySet` is another useful helper method for defining fuzzy *output* variables. It follows the same positional pattern as `GenerateCrossfadeFuzzySet`, where each numeric argument corresponds to one output label in the enum, ordered by declaration. Each value represents the crisp X-axis location, or *Representative Value*, of that linguistic output category. Optionally, a second parallel list of Degree of Membership (DoM) heights may be provided, matching the order and length of the first list. If this second list is omitted, all DoM heights default to 1.0. Discrete outputs defined in this way are particularly well suited for fuzzy logic systems using Max–Average defuzzification, where each output label contributes its representative value (RV) weighted by its activation degree. Since the RV fully defines the crisp contribution of each label, specifying additional shape details offers no benefit; only the RV and its final degree matter to the defuzzified result.

**ASCII visualization.** Slash segments represent rising or falling membership slopes, underscores and dashes represent plateaus, and the labeled keys (A–F) mark breakpoints used in the term summary below. You can visualize any fuzzy set in the Console:

```
Debug.Log(RenderFuzzySetAscii<FzOutputHeadRotX>(headRotSet));
```

Example output:

```
__________-                          /-                          /__________
           \-                    /--  \--                    /-
             \-                /-        \-                /-
               \--          /--            \--          /--
                  \-     /-                    \-     /-
                    \--/-                        \-/--
                    /--\-                        /-\--
                  /-     \-                    /-     \-
                /--          \--            /--          \--
              /-                \-        /-                \-
            /-                    \--  /--                    \-
_____________________________________________________________________________
A        B                           CD                            E        F
Key: A: -2.5, B: -1.875, C: -0.01, D: 0.01, E: 1.875, F: 2.5
Terms:
Negative: Shoulder(A, B, C, F)  H=[1, 0]  RV=-2.5
Zero:     Trapezoid(B, C, D, E) H=[0, 1, 0] RV=0
Positive: Shoulder(A, D, E, F)  H=[0, 1]  RV=2.5
Domain: [-2.5, 2.5]
```

### Creating Fuzzy Rules

Rules connect input conditions (antecedents) to output actions (consequents). Using the Domain Specific Language (DSL):

```
FuzzyRule<FzOutputHeadRotX>[] rules = new FuzzyRule<FzOutputHeadRotX>[] {
    If(FzInputBallPosZ.Negative).Then(FzOutputHeadRotX.RotPosDir),
    If(FzInputBallPosZ.Zero)    .Then(FzOutputHeadRotX.NoRot),
    If(FzInputBallPosZ.Positive).Then(FzOutputHeadRotX.RotNegDir),

    If(FzInputBallVelZ.Negative).Then(FzOutputHeadRotX.RotPosDir),
    If(FzInputBallVelZ.Zero)    .Then(FzOutputHeadRotX.NoRot),
    If(FzInputBallVelZ.Positive).Then(FzOutputHeadRotX.RotNegDir),
};
```

Group rules per-output into a rule set. This is necessary per the API for evaluation.

```
return new FuzzyRuleSet<FzOutputHeadRotX>(headRotSet, rules);
```

### About Combs Method and Fuzzy Conjunctions.

Conceptually, one might begin by writing rules such as:

> *If the ball is in the Negative Z direction and its Z velocity is Negative, then rotate the head around the X-axis in the Positive direction.*

and another:

> *If the ball is in the Negative Z direction and its Z velocity is Positive, then rotate around X very little or not at all.*

Combinatorially, we could flesh this out for all input conjunctions, but that quickly becomes unwieldy. The **Combs method** (Combs & Andrews, 1998) reduces an n-input fuzzy rule base to a set of single-input rules that are aggregated during evaluation, dramatically lowering rule count while maintaining smooth output behavior.

Rather than explicitly defining a full grid of multi-input rules, the Combs method separates each antecedent (input variable) into its own one-dimensional rule contribution and combines them mathematically during evaluation. The result approximates a complete rule table but with far fewer explicitly declared rules. This approach approximates multi-input interactions when the input influences are separable. For non-separable or sharply conjunctive cases, use explicit compound antecedents such as `If(And(A, B))`.

In this framework, the Combs-style blending is *achieved through the rule merger and weighted-average defuzzification*, which collectively reproduce Combs-style aggregation. Multiple fuzzy rules contribute partial activations to the same output fuzzy variable, and these activations are merged and averaged according to their degrees of membership. The final crisp control value thus reflects a continuous weighted combination of all relevant fuzzy rules without requiring you to manually enumerate every logical conjunction.

In general, a Combs-method approach is a good starting point. However, if a more sharply conjunctive response is desired, you can explicitly define compound antecedents using expressions such as `If(And(A, B))`. Combs assumes separable input influences; conjunctive interactions must be specified directly when needed.

### Initialization (Start Phase)

Construct fuzzy sets and rule sets once in `Start()`, and allocate your working value containers.

(In Unity, it is generally preferable to perform initialization in `Start()` rather than `Awake()`, since some setup may depend on other GameObjects that are not yet initialized during `Awake()`. This ensures proper ordering and reduces the risk of null references.)

```
void Start() {
    ...
    // Fuzzy init
    inputBallPosXSet = GetBallPosXSet();
    inputBallPosZSet = GetBallPosZSet();

    inputBallVelXSet = GetBallVelXSet();
    inputBallVelZSet = GetBallVelZSet();

    outputHeadRotXSet = GetHeadRotXSet();
    outputHeadRotZSet = GetHeadRotZSet();

    headRotXRuleSet = GetHeadRotXRuleSet(outputHeadRotXSet);
```

```
    headRotZRuleSet = GetHeadRotZRuleSet(outputHeadRotZSet);
}
```

### Runtime Evaluation (Update Phase)

Each frame, fuzzify crisp inputs into a shared `FuzzyValueSet`, then evaluate and apply rules with `ApplyFuzzyRules(...)`. The helper handles rule evaluation, merging, defuzzification, and assignment to the agent's controls.

```
// Compute crisp measurements
var currBallPosZ = ball.transform.position.z
    - topCenterOfHead.transform.position.z; // distance in only the z dimension

// Fuzzify inputs (example)
inputBallPosZSet.Evaluate(currBallPosZ,  fzInputValueSet);
inputBallVelZSet.Evaluate(m_BallRb.linearVelocity.z,  fzInputValueSet);

// Evaluate fuzzy rules and apply defuzzified results to control inputs
ApplyFuzzyRules<FzOutputHeadRotX, FzOutputHeadRotZ>(
    headRotXRuleSet,
    headRotZRuleSet,
    fzInputValueSet,
    out var headRotXRuleOutput,
    out var headRotZRuleOutput,
    ref mergedHeadRotX,
    ref mergedHeadRotZ,
    out var crispHeadRotXVal,
    out var crispHeadRotZVal
);
```

The `ApplyFuzzyRules` method performs the complete fuzzy inference cycle each frame. It evaluates all active rules, merges overlapping contributions to the same output fuzzy variable, and then defuzzifies to produce crisp control outputs. The resulting crisp values are applied to the head rotation controller. The additional `out` and `ref` parameters expose intermediate merged values and individual rule outputs, which can be logged for debugging or visualized to study how different input memberships interact during rule evaluation.

Note that the crisp outputs do not instantly set the head rotation. Instead, they feed into a simple physical simulation with rotational acceleration limits and a cap on maximum angular velocity. This preserves smooth, realistic head motion even when fuzzy rules change rapidly.

### Conclusion

The FLBall3DAgent example demonstrates the core loop of a fuzzy control system in Unity: define linguistic variables, construct fuzzy sets, author rules, fuzzify inputs, evaluate and defuzzify, and apply the crisp outputs to drive behavior. Once you are comfortable with this workflow, you can extend it to the Racetrack assignment, where fuzzy rules govern throttle and steering instead of head rotation. The same pattern of crossfading inputs, discrete outputs, and smooth aggregation applies directly.